

סקריפט ה-DNS Failover – הסבר שורה אחר שורה

מעבר רצוף על שלושת הסקריפטים שמנהלים את רשומת ה-DNS של SolarWinds: כל מקטע שורות, מה הוא עושה, ולמה הוא כתוב כך.

תוכן העניינים

1. איך לקרוא את המסמך, ורקע על המערכת
2. סקריפט א' – הגרסה המקורית, שורה אחר שורה
3. סקריפט ב' – הגרסה המתוקנת, שורה אחר שורה
4. סקריפט ג' – בלי Alteon ובלי שרתי web, שורה אחר שורה
5. השוואה, מצב הבדיקות ומה לאמת לפני הרצה

1. איך לקרוא את המסמך, ורקע על המערכת

מבנה המסמך

שלושת הפרקים הבאים עוברים על שלושת הסקריפטים **ברצף מלא, מהשורה הראשונה עד האחרונה**. הקוד מחולק למקטעים, וכל מקטע מוצג כך:

• **טווח השורות** – מופיע בתווית משמאל לכותרת המקטע.

• **הקוד עצמו** – בדיוק כפי שהוא בקובץ.

• **ההסבר** – מה הקוד עושה, ולמה הוא נכתב דווקא כך.

המקטעים **רצופים ואינם חופפים**: כל מקטע מתחיל בשורה שאחרי סוף הקודם. אין דילוגים ואין שורה שמופיעה פעמיים – גם שורות ריקות וסוגריים סוגרים נכללים בטווח של המקטע שהם שייכים לו.

הקוד במסמך נשלף אוטומטית מקבצי המקור לפי טווחי השורות, ולא הועתק ידנית. מחולל המסמך גם מוודא, בכל הרצה, שכל שורה בכל קובץ נכללת בדיוק במקטע אחד – ונעצר בשגיאה אם נוצר פער או חפיפה. לכן הקוד המוצג כאן זהה לקוד שברישו, ומספרי השורות מדויקים.

סה"כ 63 מקטעים המכסים 762 שורות קוד.

שלושת הסקריפטים

קובץ	שורות	מקטעים	איך הוא מחליט
DNS.SetSolarWindsRecordToActiveLB.ps1	111	18	תשאול SNMP של ה-Load Balancer על מצב שרתי ה-web. הגרסה המקורית, לא נערכה.
DNS.SetSolarWindsRecordToActiveLB.v2.ps1	411	33	אותה החלטה בדיוק, עם טיפול בשגיאות, לוג, ולידציה ו-WhatIf.
DNS.SetSolarWindsRecordToActiveServer.ps1	240	12	בלי SNMP, בלי שרתי web ובלי LB: מקבל מ-SolarWinds את כתובת השרת שהפך לפעיל, והרשומה עוקבת אחריו.

רקע: מה המערכת עושה

ל-SolarWinds יש שם DNS אחד שכולם בארגון משתמשים בו – `OurSolar.OurZone`. השם הזה חייב להצביע תמיד על כתובת שבאמת מגישה את הממשק. יש שני מרכזי נתונים, וכל הסקריפטים עונים על אותה שאלה: **לאיזו כתובת הרשומה צריכה להצביע עכשיו?**

רכיב	PrimaryDC	SecondaryDC	תפקיד
שרת SolarWinds	10.10.10.1	10.10.20.1	מי שמריץ את הסקריפט. בסקריפט ג' הוא גם יעד ברירת המחדל של הרשומה.
Load Balancer – כתובת שירות	10.10.10.10	10.10.20.10	היעד שנכתב לרשומת ה-DNS בסקריפטים א' ו-ב'. בסקריפט ג' אין LB בנתיב, והרשומה מצביעה על שרת ה-SolarWinds עצמו.
Load Balancer – כתובת ניהול	10.10.10.100	10.10.20.100	הכתובת שנשאלת ב-SNMP. רלוונטית רק לסקריפטים א' ו-ב'; סקריפט ג' אינו נוגע בה.
רשומת DNS	OurSolar.OurZone		רשומת A יחידה. זו הרשומה שכל הסקריפטים משנים.

שתי כתובות שונות לאותו LB. כתובת השירות (`10`) היא מה שנכתב לרשומה, וכתובת הניהול (`100`) היא מה שנשאל ב-SNMP. החלפה ביניהן תיצור סקריפט שנראה תקין אבל מפנה תעבורה לכתובת שאינה מגישה את השירות.

2. סקריפט א' – הגרסה המקורית, שורה אחר שורה

הקובץ `DNS.SetSolarWindsRecordToActiveLB.ps1`, 111 שורות, ב-18 מקטעים רצופים. זוהי הגרסה שתומללה מההדפסה ולא נערכה. היא נשמרה בריפו כפי שהיא, כדי שאפשר יהיה להשוות.

1-3 פתיחת הפונקציה

```
function dns_ip{  
# this function get me the dns ip based on the server's dns
```

הפונקציה `dns_ip` נפתחת. ההערה מתארת את הכוונה: למצוא את כתובת שרת ה-DNS שהשרת הזה עצמו משתמש בו. הרעיון נכון – במקום לקבוע כתובת קשיחה בקוד, לגזור אותה מהגדרות הרשת של המכונה. הבעיות מתחילות במימוש.

4-5 שליפת הגדרות כרטיסי הרשת

```
$networks = get-wmiobject -Class Win32_NetworkAdapterConfiguration
```

`Get-WmiObject` מחזיר אובייקט לכל כרטיס רשת במכונה. שתי הערות:

- הפקודה **מיושנת**. מאז PowerShell 3.0 החליף אותה `Get-CimInstance`, וב-PowerShell 7 הוסרה לגמרי. סקריפט שמשתמש בה לא ירוץ שם.
- **אין סינון**. מוחזרים גם כרטיסים מנוטרלים, מנותקים, ומתאמי וירטואליזציה. כל אחד מהם עלול לתרום כתובת DNS ישנה לרשימה.

6-11 שתי הלולאות

```
foreach($n in $networks){  
  
    $network = $n.DNSServerSearchOrder  
  
    foreach($ip in $network){
```

הלולאה החיצונית עוברת על הכרטיסים; הפנימית על `DNSServerSearchOrder` – מערך כתובות ה-DNS המוגדרות על אותו כרטיס, לפי סדר. המבנה הגיוני: לעבור על המועמדים עד שנמצא אחד תקין.

```
try{
    test-connection -ComputerName $ip -Count 1

    $new = $ip

    return [String]$new
}

catch{continue}
```

זהו המקטע החשוב ביותר בסקריפט המקורי. שלוש בעיות נפרדות מצטברות כאן:

- 1. הפלט של Test-Connection דולף.** שורה 14 קוראת לפקודה בלי לתפוס את הפלט שלה למשתנה ובלי לשלוח אותו ל-`Out-Null`. ב-PowerShell, כל ערך שפונקציה מייצרת ולא נבלע – נכנס לערך ההחזרה שלה. התוצאה: הפונקציה מחזירה **מערך בן שני איברים** – אובייקט ה-ping ואחריו המחרוזת – ולא מחרוזת בודדת כפי שכתוב בשורה 18.
- 2. ה- catch אף פעם לא נכנס לפעולה.** כשל של `Test-Connection` מייצר שגיאה **לא מפסיקה**. רק שגיאה מפסיקה מפעילה `catch`. בלי `-ErrorAction Stop`, גוש ה-`catch` בשורה 21 הוא קוד מת.
- 3. ה- return אינו מותנה.** שורה 18 רצה תמיד, גם כשה-ping נכשל. המשמעות: הפונקציה תמיד מחזירה את כתובת ה-DNS הראשונה ברשימה, בין אם היא חיה ובין אם לא. כל הלולאה, כל ה-`try` וכל ה-`continue` – חסרי משמעות מעשית.

```
}
}
}
```

סגירת הלולאות והפונקציה. שימו לב שאין כאן `return` למקרה שאף כתובת לא נמצאה – מסלול שממילא בלתי ניתן להגעה בגלל ה-`return` הלא-מותנה שלמעלה.

```
#####
# those are all the details dont touch it if you didnt change the server ip, dns zone or the name
$active_server = $args[0]
```

ההערה מסמנת את אזור הקונפיגורציה. שורה 30 היא נקודת הכניסה היחידה של הסקריפט: `$args[0]` – הארגומנט הראשון שהועבר בשורת הפקודה.

אין כאן **שום ולידציה**: לא בדיקה שהועבר ארגומנט, לא בדיקה שהוא כתובת IP, ולא בדיקה שהוא אחת משתי הכתובות שהסקריפט מכיר. ארגומנט חסר או שגוי יוביל לכך ששני תנאי ה-`if` הגדולים לא יתקיימו, והסקריפט יסתיים **בהצלחה** **מדומה בלי לעשות דבר** – הכשל השקט הגרוע ביותר בסקריפט `failover`.

כתובות המערכת

32–40

```
$solarwinds_name = "OurSolar"

$SecondaryDC_solar_ip = "10.10.20.1"
$PrimaryDC_solar_ip = "10.10.10.1"

$LB_SecondaryDC = "10.10.20.10"
$LB_PrimaryDC = "10.10.10.10"

$dns_server_ip = dns_ip
```

שם הרשומה ושם הכתובות שהסקריפט מכיר, מסודרות בשלושה זוגות:

- `$PrimaryDC_solar_ip` / `$SecondaryDC_solar_ip` – שרתי ה-SolarWinds. מולם מושווה `$active_server` כדי לדעת עבור איזה מרכז נתונים הריצה מתבצעת.
 - `$LB_PrimaryDC` / `$LB_SecondaryDC` – כתובות השירות של ה-Load Balancer. אחת מהן תיכתב לרשומת ה-DNS.
- הזוג השלישי – כתובות הניהול של ה-LB לתשאול SNMP – אינו כאן אלא קבוע בתוך הקוד עמוק יותר, בשורות 59 ו-89. פיצול מיותר: חלק מהכתובות בגוש ההגדרות וחלק מוסתרות בלוגיקה.

השורה ששוברת הכול ⚠

41–45

```
$dns_server_ip = $dns_server_ip.IPV4Address.IPAddressToString
$dns_server_ip

$zone = "OurZone"
```

שורה 41 קוראת ל-`dns_ip`. שורה 43 היא התוצאה הישירה של הבאג שראינו בשורות 12–21.

אם `dns_ip` הייתה מחזירה מחרוזת כפי שכתוב בה, אז `IPV4Address`. על מחרוזת היה מחזיר `$null` והכול היה קורס. הסיבה היחידה ששורה 43 עובדת היא שהפונקציה מחזירה בפועל **מערך** שכולל את אובייקט ה-`ping`, ו-PowerShell שולף את המאפיין מהאיבר שיש לו אותו.

כלומר: **הקוד עובד בזכות באג, לא למרותו**. ברגע שהמבנה של `Test-Connection` משתנה – וב-7 PowerShell הוא אכן שונה ואין בו `IPV4Address` – המשתנה מתאפס ל-`$null` וכל פקודות ה-DNS נכשלות.

שורה 44 היא שורה בודדת עם שם משתנה. ב-PowerShell זו הדפסה למסך – שריד דיבוג.

```
$oldObj = Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName
$zone

$ip = $oldObj.RecordData.IPv4Address.IPAddressToString

#####
```

שורה 48 שולפת את הרשומה, ושורה 50 חולצת ממנה את הכתובת הנוכחית לצורך השוואה בהמשך. שתי בעיות:

1. **אין A -RRType**. הפקודה תחזיר כל רשומה בשם הזה, מכל סוג. אם קיימות שתי רשומות (round-robin), A, \$oldObj יהיה מערך – וההשמה למאפיין בהמשך תיכשל בהודעה לא ברורה.
2. **אין בדיקת הצלחה**. אם \$dns_server_ip הוא \$null (ראו שורה 43), הפקודה נכשלת – והסקריפט ממשיך לרוץ עם \$oldObj ריק.

```
if ($active_server -eq $SecondaryDC_solar_ip){
```

מפריד ויזואלי בלבד. מכאן מתחילה לוגיקת ההחלטה.

```
# if its on PrimaryDC change it to SecondaryDC
$SNMP = New-Object -ComObject olePrn.oleSNMP
# Get status of web servers on SecondaryDC
$SNMP.Open('10.10.20.100', 'public',2,1000)
$resultServerOne = $SNMP.Get('.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.54')
$resultServerTwo = $SNMP.Get('.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.55')
$SNMP.Close()
```

אם השרת הפעיל הוא זה של SecondaryDC, נפתח session SNMP מול כתובת הניהול של ה-LB המקומי (10.10.20.100, קבועה בקוד) ונקראים שני ערכים – אחד לכל שרת web.

⚠ **אין כאן try/catch**. זו הנקודה המסוכנת ביותר בסקריפט כולו: אם ה-LB אינו זמין, `$SNMP.Open()` יזרוק חריגה, הסקריפט ימות באמצע, **לא יתבצע failover, ואף אחד לא יידע**. כלומר בדיוק בתרחיש שלשמו הסקריפט נכתב – הוא לא עובד. בנוסף `$SNMP.Close()` בשורה 62 לא ירוץ, ואובייקט ה-COM ידלוף.

הפרמטרים של `Open`: כתובת, community string (public – ברירת מחדל ולא המלצה), גרסה 2, ו-timeout במילישניות.

```
# If one web server goes down and LB in PrimaryDC, change to SecondaryDC LB
if (($resultServerOne -eq 0 -or $resultServerTwo -eq 0) -and $ip -ne $LB_SecondaryDC) {
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone
    Write-Host aaaaa
    $NewObj = Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name "OurSolar" -ZoneName
$zone
    $NewObj.RecordData.IPV4Address = [System.Net.IPAddress]::Parse($LB_SecondaryDC)
    Set-DnsServerResourceRecord -ZoneName $zone -ComputerName $dns_server_ip -OldInputObject
$soldobj -NewInputObject $NewObj
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone
}
```

התנאי: **לפחות אחד** מהערכים שווה ל-0, וגם הרשומה עדיין לא מצביעה על ה-LB המקומי. אם כן – הרשומה מועברת ל-LB של SecondaryDC.

החלק השני של התנאי, `-and $ip -ne $LB_SecondaryDC`, הוא דווקא **נכון וטוב**: הוא מונע כתיבה מיותרת כשהרשומה כבר במקום הנכון. זו בדיקת אידמפוטנטיות תקינה.

שורה 66 – `Write-Host aaaaa` – שריד דיבוג. שורות 65 ו-70 קוראות את הרשומה ומדפיסות אותה בלי לשמור לכלום; גם הן שרידי דיבוג.

שורה 67 שולפת `$NewObj` בקריאה נפרדת. זה **נכון ונדרש**: `Set-DnsServerResourceRecord` דורשת שני אובייקטים נפרדים, המקורי והערוך. שימו לב ששם הרשומה כאן קבוע כ- "OurSolar" במקום להשתמש ב- `$solarwinds_name` – אי-עקביות שתשבור את הסקריפט אם ישנו את שם הרשומה בגוש ההגדרות בלבד.

```
# If both web servers goes down and LB not in of SecondaryDC, change to PrimaryDC LB
elseif (($resultServerOne -ne 0 -and $resultServerTwo -ne 0) -and $ip -ne $LB_PrimaryDC) {
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone
    Write-Host aaaaa
    $NewObj = Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name "OurSolar" -ZoneName
$zone
    $NewObj.RecordData.IPV4Address = [System.Net.IPAddress]::Parse($LB_PrimaryDC)
    Set-DnsServerResourceRecord -ZoneName $zone -ComputerName $dns_server_ip -OldInputObject
$soldobj -NewInputObject $NewObj
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone
}
```

התנאי ההפוך: **שני** הערכים שונים מ-0, וגם הרשומה עדיין לא על ה-LB המרוחק. במקרה כזה הרשומה עוברת ל-LB של PrimaryDC – כלומר failover.

מכאן נגזרת המשמעות של הערך 0. אם 0 היה מסמן 'השרת נפל', הענף הראשון היה מבצע failover כששרת אחד נופל, והענף הזה היה משאיר את התעבורה במקום כששניהם תקינים – חסר היגיון. הקריאה היחידה שמתיישבת עם המבנה היא השרת למעלה = 0: לפחות אחד למעלה → נשארים מקומית; כולם נפלו → עוברים.

ההערות בקוד עצמו מתויגות הפוך בחלק מהמקומות ולכן אינן אמינות – הקוד הוא המקור.

שורות 74–80 הן שכפול כמעט מדויק של שורות 65–71, עם שינוי כתובת יעד אחד.

82–84 סגירת בלוק SecondaryDC

```
}
```

סגירת ה-`elseif` וה-`if` הגדול. שימו לב: אם אף אחד משני התנאים לא מתקיים (למשל הרשומה כבר במקום הנכון) – לא נכתב כלום ולא נרשם כלום. התנהגות נכונה, אבל שקטה לחלוטין.

85–92 בלוק PrimaryDC – תשאול SNMP

```
if ($active_server -eq $PrimaryDC_solar_ip){  
    # if its on SecondaryDC change it to PrimaryDC  
    $SNMP = New-Object -ComObject olePrn.oleSNMP  
    # Get status of web servers on PrimaryDC  
    $SNMP.Open('10.10.10.100', 'public', 2, 1000)  
    $resultServerOne = $SNMP.Get('.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.51')  
    $resultServerTwo = $SNMP.Get('.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.52')  
    $SNMP.Close()  
}
```

אותו מבנה בדיוק, עבור מרכז הנתונים השני: כתובת ניהול `10.10.10.100` ו-`OID`ים של שרתי ה-`web` של `PrimaryDC`. כל הבעיות מהבלוק הקודם חוזרות כאן זהות – אין `try/catch`, אין שחרור מסודר, והכתובת קבועה בקוד.

93–101 בלוק PrimaryDC – ענף ראשון

```
# If both web servers goes down and LB not in SecondaryDC, change to PrimaryDC LB  
if (($resultServerOne -eq 0 -or $resultServerTwo -eq 0) -and $ip -ne $LB_PrimaryDC) {  
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone  
    Write-Host aaaaa  
    $NewObj = Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name "OurSolar" -ZoneName  
    $zone  
    $NewObj.RecordData.IPV4Address = [System.Net.IPAddress]::Parse($LB_PrimaryDC)  
    Set-DnsServerResourceRecord -ZoneName $zone -ComputerName $dns_server_ip -OldInputObject  
    $oldobj -NewInputObject $NewObj  
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone  
}
```

→ ה-`LB` המקומי, שהוא הפעם זה של `PrimaryDC`. הכלל הזה לחלוטין לזה של הבלוק הקודם – 'לפחות אחד למעלה, הישאר מקומי'. מה שמתחלף זו רק זהות ה'מקומי' וה'מרוחק', לא ההיגיון.

זו נקודה שחשוב להדגיש: אין כאן לוגיקה הפוכה בין שני הבלוקים. הם סימטריים ותקינים. רק ההערות מטעות.

```
# If one web server goes down and LB in PrimaryDC, change to SecondaryDC LB
elseif (($resultServerOne -ne 0 -and $resultServerTwo -ne 0) -and $ip -ne $LB_SecondaryDC) {
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone
    Write-Host aaaaa
    $NewObj = Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name "OurSolar" -ZoneName
$zone
    $NewObj.RecordData.IPV4Address = [System.Net.IPAddress]::Parse($LB_SecondaryDC)
    Set-DnsServerResourceRecord -ZoneName $zone -ComputerName $dns_server_ip -OldInputObject
$oldobj -NewInputObject $NewObj
    Get-DnsServerResourceRecord -ComputerName $dns_server_ip -Name $solarwinds_name -ZoneName $zone
}
```

שני השרתים נפלו → failover ל-LB של SecondaryDC. שכפול רביעי ואחרון של אותן שמונה שורות.

סה"כ: **ארבעה בלוקים כמעט זהים** בשורות 65-71, 74-80, 95-101 ו-104-110. כל תיקון עתידי – הוספת לוג, טיפול בשגיאה, שינוי TTL – חייב להיעשות ארבע פעמים, וכל פעם שנשכחת היא באג. שימו לב גם לשורה 104, שהזחתה שונה משאר השורות בבלוק – סימן להעתקה ידנית.

```
}
```

סגירת הבלוק האחרון. אין כאן `exit` עם קוד יציאה, ולכן הקורא – SolarWinds, Task Scheduler או כל מתזמן אחר – לא יכול לדעת אם הריצה הצליחה, נכשלה, או לא עשתה דבר.

3. סקריפט ב' – הגרסה המתוקנת, שורה אחר שורה

הקובץ `DNS.SetSolarWindsRecordToActiveLB.v2.ps1`, 411 שורות, ב-33 מקטעים רצופים. אותה החלטה עסקית כמו בגרסה המקורית, עם טיפול בשגיאות, לוג, ולידציה ותצוגה מקדימה.

1-2 הצהרת גרסה

```
#Requires -Version 5.1
```

`#Requires` היא הוראה למנוע ההרצה, לא הערה. אם הסקריפט מורץ תחת PowerShell ישן מ-5.1, הוא נעצר מיד עם הודעה ברורה במקום להיכשל באמצע על פקודה חסרה. שורת הגנה ראשונה.

3-18 בלוק העזרה – תיאור

```
<#  
.SYNOPSIS  
    Points the SolarWinds DNS A record at the load balancer of the data centre  
    that is currently able to serve it.  
  
.DESCRIPTION  
    The script is meant to run as an alert action on the SolarWinds server that  
    has just become active. It is told which SolarWinds server it is running  
    for, polls that data centre's web servers through the local load balancer  
    over SNMP, and repoints the A record at the local load balancer while the  
    data centre is serving, or at the other data centre's load balancer once  
    every web server there has stopped answering.  
  
    Nothing is written when the record already holds the wanted address, so the  
    script is safe to run on a schedule.
```

בלוק `<# ... #>` בראש סקריפט הוא **עזרה מובנית** של PowerShell. הוא אינו הערה רגילה: הרצת `Get-Help .\script.ps1 -Full` מציגה אותו מעוצב, בדיוק כמו העזרה של כל פקודה מובנית.

`.SYNOPSIS` הוא משפט אחד שאומר מה הסקריפט עושה, ו-`.DESCRIPTION` מרחיב: מתי הוא רץ, מה הוא בודק, ומה הוא מחליט. שורות 16-17 מציגות את תכונת האידמפוטנטיות – שאין כתיבה כשאין מה לשנות – וזו הסיבה שמותר להריץ אותו מתוזמן.

.PARAMETER ActiveServer

The address of the SolarWinds server this run is for. It has to match either PrimaryDcSolarWindsIp or SecondaryDcSolarWindsIp, otherwise the script fails instead of quietly doing nothing.

.PARAMETER DnsServer

The DNS server to read the record from and write it back to. It has to hold a writable copy of the zone. When it is left out the script walks the DNS servers configured on this host and takes the first one that answers a ping.

.PARAMETER HealthyValue

The SNMP value that means "this web server is up". Everything else counts as down. See the note at the bottom of this help before changing it.

.PARAMETER PollCount

How many times a data centre is polled before it is declared down. This is what keeps a single bad reading from moving the record.

.PARAMETER SnmpFailureMode

What to do when none of the polls get an answer at all. 'Down' treats an unreachable load balancer as a dead data centre and fails over, which is the behaviour the original script had. 'Unknown' leaves the record alone and exits with code 2, which is the safer choice if SNMP is flaky.

.PARAMETER RecordTtlSeconds

When greater than zero, the TTL written onto the record. A failover only takes effect once the old TTL has expired everywhere, so a low value (30-60) is worth setting here.

.PARAMETER לכל פרמטר שדורש הסבר. לא כולם מתועדים כאן – רק אלה שההתנהגות שלהם אינה מובנת מאליה מהשם.

שימו לב לשורות 29–31: התיעוד של **HealthyValue** מפנה במפורש לאזהרה שבסוף הבלוק. זו ההנחה היחידה בסקריפט שאי אפשר לאמת מקריאת הקוד.

שורות 37–41 מתעדות את **SnmpFailureMode**, שהוא ההבדל המהותי מול הסקריפט המקורי: בחירה מודעת בין 'אי-מענה' נחשב נפילה לבין 'אי-מענה' משאיר את הרשומה במקום.

.EXAMPLE

```
.\DNS.SetSolarWindsRecordToActiveLB.v2.ps1 -ActiveServer 10.10.10.1 -WhatIf
```

Shows what would change without touching DNS.

.EXAMPLE

```
.\DNS.SetSolarWindsRecordToActiveLB.v2.ps1 -ActiveServer 10.10.20.1 -Verbose
```

שתי דוגמאות הרצה. הראשונה משתמשת ב-`-WhatIf` – הדרך הבטוחה לבדוק את הסקריפט בלי לשנות DNS אמיתי. דוגמה טובה בעזרה חוסכת קריאה של הקוד.

56–73 בלוק העזרה – הערות תפעול

.NOTES

Exit codes: 0 done (changed or nothing to change), 1 failed, 2 the state of the data centre could not be determined and the record was left alone.

Call it with `-File`, not `-Command`:

```
powershell.exe -NoProfile -File DNS.SetSolarWindsRecordToActiveLB.v2.ps1 -ActiveServer 10.10.10.1
```

A parameter that fails validation stops the script before it runs, and only `-File` turns that into exit code 1. Under `-Command` the caller reads a stale `$LASTEXITCODE` instead, so a typo in the arguments looks like a clean run.

`HealthyValue` defaults to 0 because that is what the original script compared against. Confirm it against the devices before relying on this in production: if 0 does not in fact mean "up" on your load balancers, every decision the script makes is inverted.

#>

שלושה דברים שמפעיל הסקריפט חייב לדעת:

1. **קודי היציאה** (שורות 57–58) – 0 הצליח, 1 נכשל, 2 לא ניתן היה לקבוע.
2. **חובה להפעיל עם `-File`** (שורות 60–66). זו נקודה עדינה וחשובה: פרמטר שנכשל בוולידציה עוצר את הסקריפט **לפני** שגופו רץ. רק `-File` מתרגם כישלון כזה לקוד יציאה 1. תחת `-Command` הקורא מקבל ערך `$LASTEXITCODE` ישן, ולכן טעות הקלדה בפרמטרים תיראה כמו ריצה מוצלחת.
3. **האזהרה על `HealthyValue`** (שורות 68–71) – אם 0 אינו מסמן שרת תקין, כל ההחלטות מתהפכות.

74–79 הצהרת הפרמטר הראשון

```
[CmdletBinding(SupportsShouldProcess)]
param(
    [Parameter(Mandatory, Position = 0)]
    [ValidateNotNullOrEmpty()]
    [string] $ActiveServer,
```

`[CmdletBinding(SupportsShouldProcess)]` הופך את הסקריפט ל"פונקציה מתקדמת": הוא מקבל אוטומטית את `-ErrorAction`, `-Verbose`, וחשוב מכל – `-WhatIf`, שיגיע לידי ביטוי בשורה 397.

`$ActiveServer` מוצהר `Mandatory` עם `ValidateNotNullOrEmpty`. זהו התיקון הישיר ל-`$args[0]` של הסקריפט המקורי: ארגומנט חסר או ריק נעצר בשלב קישור הפרמטרים, לא בשקט באמצע הריצה. `Position = 0` משאיר תאימות למי שמעביר את הכתובת בלי לנקוב בשם הפרמטר.

```
[ValidateNotNullOrEmpty()]
[string] $RecordName = 'OurSolar',

[ValidateNotNullOrEmpty()]
[string] $ZoneName = 'OurZone',

[string] $DnsServer,
```

שלושת הערכים שהיו קבועים בקוד המקורי הופכים לפרמטרים. `$DnsServer` הוא היחיד בלי `ValidateNotNullOrEmpty` – במכון: השארנו ריקה היא בחירה חוקית שמפעילה את האיתור האוטומטי בשורה 362.

```
[ValidateNotNullOrEmpty()]
[string] $PrimaryDcSolarWindsIp = '10.10.10.1',

[ValidateNotNullOrEmpty()]
[string] $SecondaryDcSolarWindsIp = '10.10.20.1',

[ValidateNotNullOrEmpty()]
[string] $PrimaryDcLoadBalancerIp = '10.10.10.10',

[ValidateNotNullOrEmpty()]
[string] $SecondaryDcLoadBalancerIp = '10.10.20.10',
```

ארבע הכתובות שהיו בגוש ההגדרות של הסקריפט המקורי. ההבדל: עכשיו אפשר לשנות כל אחת מהן משורת הפקודה, בלי לערוך את הקובץ. זה מה שמאפשר לבדוק את הסקריפט מול סביבת מעבדה עם כתובות אחרות.

```
[ValidateNotNullOrEmpty()]
[string] $PrimaryDcSnmpHost = '10.10.10.100',

[ValidateNotNullOrEmpty()]
[string] $SecondaryDcSnmpHost = '10.10.20.100',
```

הכתובות שבסקריפט המקורי היו **מוסתרות עמוק בתוך הלוגיקה** (שורות 59 ו-89 שם) עולות עכשיו לרמת הפרמטרים, לצד שאר הכתובות.

חשוב לא לבלבל: אלה כתובות ה**ניהול** של ה-LB (100), ולא כתובות השירות (10). שנכתבות לרשומת ה-DNS. החלפה ביניהן תיצור סקריפט שנראה תקין אבל מפנה תעבורה לכתובת שאינה מגישה את השירות.

```
[string[]] $PrimaryDcWebServerOid = @(
    '1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.51',
    '1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.52'
),

[string[]] $SecondaryDcWebServerOid = @(
    '1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.54',
    '1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.55'
),
```

ארבעת ה-OLD-ים, שניים לכל מרכז נתונים. הם תחת 1.3.6.1.4.1.1872 – מרחב ה-enterprise של Radware. הספרות בסוף אינן שרירותיות: הן מקודדות את שמות השרתים כתווי ASCII. 52.52 הוא "44", ו-57.51 הוא "93". מכאן נובעת מסקנה תפעולית: **אם שרת web מוחלף או משנה שם, ה-OLD משתנה איתו** והסקריפט יקרא ערך של שרת שאינו קיים. לכן הם פרמטרים – ולכן הם מערכים, כך שאפשר להוסיף שרת שלישי בלי לגעת בקוד.

```
[ValidateNotNullOrEmpty()]
[string] $SnmpCommunity = 'public',

[int] $HealthyValue = 0,

[ValidateRange(1, 10)]
[int] $PollCount = 3,

[ValidateRange(0, 60)]
[int] $PollDelaySeconds = 5,

[ValidateSet('Down', 'Unknown')]
[string] $SnmpFailureMode = 'Down',

[ValidateRange(0, 86400)]
[int] $RecordTtlSeconds = 0,

[string] $LogPath = (Join-Path -Path $env:ProgramData -ChildPath 'SolarWinds\dns-failover.log')
)
```

כאן מרוכזות כל ההחלטות שהיו קבועות בקוד המקורי:

- `$SnmpCommunity` – היה 'public' קבוע. ברירת מחדל, לא המלצה.
- `$HealthyValue` – הערך שמסמן שרת תקין. **ההנחה הקריטית.**
- `$PollCount` ו-`$PollDelaySeconds` – הגנת ה-flapping. מונע ערכים חסרי משמעות.
- `$SnmpFailureMode` – `ValidateSet` מגביל לשתי המחרוזות החוקיות, כך שטעות כתיב נתפסת בקישור הפרמטרים.
- `$RecordTtlSeconds` – אפס פירושו 'אל תיגע ב-TTL'.

• `$LogPath` – ברירת המחדל נבנית מ- `$env:ProgramData`, המיקום המוסכם בווינדוס לנתוני יישום.

מצב קפדני וטיפול בשגיאות 135–138

```
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
```

שתי שורות שמשנות את התנהגות כל הסקריפט:

`Set-StrictMode -Version Latest` הופך שגיאות שקטות לרעשניות: פנייה למשתנה שלא הוגדר, או למאפיין שאינו קיים, נעצרת במקום להחזיר `$null`. בדיוק סוג הבאג שהפיל את הסקריפט המקורי בשורה 43 שלו.

`$ErrorActionPreference = 'Stop'` הופך שגיאות לא-מפסיקות למפסיקות. זה מה שגורם ל- `try/catch` לעבוד באמת – התיקון הישיר ל- `catch` המת בשורות 12–21 של הסקריפט המקורי.

Write-Log – חתימה ועיצוב 139–151

```
function Write-Log {
    param(
        [Parameter(Mandatory)]
        [AllowEmptyString()]
        [string] $Message,

        [ValidateSet('INFO', 'WARN', 'ERROR')]
        [string] $Level = 'INFO'
    )

    $line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
    Write-Information -MessageData $line -InformationAction Continue
}
```

פונקציית הלוג המרכזית. `$Level` מוגבל בוולידציה לשלוש רמות, כך שאי אפשר להמציא רמה חדשה בטעות.

שורה 149 בונה את השורה: חותמת זמן בפורמט קבוע, רמה מיושרת לרוחב 5 תווים (`{1,-5}`) כך שהעמודות מסתדרות בקובץ, ואז ההודעה.

שורה 150 משתמשת ב- `Write-Information` ולא ב- `Write-Host`. ההבדל מעשי: `Write-Host` כותב ישירות למסך ואי אפשר להפנות אותו, בעוד `Write-Information` עובר בזרם מסודר שמתזמן יכול לתפוס.

```
# -WhatIf:$false so that a -WhatIf run still leaves a trace of what it
# decided; the log is a record, never one of the changes being previewed.
try {
    $directory = Split-Path -Path $script:LogPath -Parent
    if ($directory -and -not (Test-Path -LiteralPath $directory)) {
        New-Item -Path $directory -ItemType Directory -Force -WhatIf:$false -Confirm:$false | Out-Null
    }
    Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -WhatIf:$false -Confirm:$false
}
catch {
    Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
}
```

`-WhatIf:$false` בשורות 157 ו-159 הוא פרט קריטי שהתגלה בבדיקות. בלעדיו, הרצה עם `-WhatIf` הייתה מחלחלת גם לפקודות הלוג: הקובץ לא היה נכתב, והמסך היה מוצף בהודעות 'What if: Performing the operation Add Content'. ההיגיון: הלוג הוא תיעוד, לא אחד השינויים שמציגים בתצוגה מקדימה. דווקא בהרצת `-WhatIf` חשוב שיישאר תיעוד של מה הסקריפט החליט.

ה-`catch` בשורות 161–163 מבטיח שכשל בכתיבה ללוג – דיסק מלא, הרשאות – לא יפיל את הסקריפט. הלוג חשוב, אבל לא יותר מהמשימה עצמה.

```
function Resolve-DnsServerAddress {
    # Walks the DNS servers configured on this host and returns the first one
    # that answers a ping. Only adapters with IP enabled are looked at, so
    # disconnected or disabled NICs cannot contribute a stale address.
    [OutputType([string])]
    param()

    $adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -Filter 'IPEnabled = True'
```

המקבילה המתוקנת ל-`dns_ip` המקורית. שלושת ההבדלים:

1. `Get-CimInstance` במקום `Get-WmiObject` – הפקודה הנתמכת.
2. `'Filter 'IPEnabled = True'` – רק כרטיסים פעילים. כרטיס מנוטרל לא יכול לתרום כתובת DNS ישנה.
3. הסינון נעשה בצד ה-CIM ולא בצד PowerShell, כלומר מסננים לפני ההעברה ולא אחריה.

```
foreach ($adapter in $adapters) {
    foreach ($address in $adapter.DNSServerSearchOrder) {
        if (Test-Connection -ComputerName $address -Count 1 -Quiet -ErrorAction SilentlyContinue) {
            Write-Log "Using DNS server $address, configured on '$($adapter.Description)'."
            return [string]$address
        }

        Write-Log "DNS server $address did not answer a ping, trying the next one." -Level WARN
    }
}

throw 'None of the DNS servers configured on this host answered a ping. Pass -DnsServer to name one explicitly.'
}
```

כאן מתוקנים שני הבאגים המרכזיים של הפונקציה המקורית:

`Test-Connection` נמצאת בתוך `if` עם `-Quiet`, שגורם לה להחזיר `$false / $true` במקום אובייקט. אין דליפת פלט, ואין `return` לא-מותנה: מחזירים רק כתובת שענתה בפועל.

`-ErrorAction SilentlyContinue` נחוץ כאן דווקא בגלל `$ErrorActionPreference = 'Stop'` משורה 137: בלעדיו, שרת DNS ראשון שאינו עונה היה זורק חריגה במקום לאפשר מעבר לבא בתור.

שורה 186 היא ההבדל החשוב ביותר: אם אף שרת לא ענה – **זריקת שגיאה מפורשת** עם הצעה לפעולה. הפונקציה המקורית הייתה מחזירה במקרה כזה `$null` בשקט.

```
function Get-WebServerHealth {
    # Reads every OID once and reports how many web servers came back healthy.
    # Answered is false only when the load balancer gave us nothing at all,
    # which is what tells a dead data centre apart from a dead SNMP path.
    param(
        [Parameter(Mandatory)][string] $SnmpHost,
        [Parameter(Mandatory)][string[]] $Oid,
        [Parameter(Mandatory)][string] $Label
    )

    $snmp = $null
    $answered = 0
    $healthy = 0
}
```

הפונקציה שמחליפה את בלוקי ה-SNMP הכפולים. ההערה בשורות 190–192 מסבירה את ההחלטה המרכזית בעיצוב שלה.

שני המונים בשורות 200–201 הם הליבה: `$answered` סופר כמה OID-ים החזירו תשובה כלשהי, ו-`$healthy` כמה מהם דיווחו על שרת תקין.

ההפרדה הזו היא כל העניין: 'כל השרתים נפלו' ולא הצלחנו לשאול' הם שני מצבים שונים לחלוטין, שמובילים להחלטות שונות. בסקריפט המקורי שניהם נראו זהים, כי `$null -eq 0` הוא `False` בדיוק כמו `0 -eq 2`.

203–215 Get-WebServerHealth – פתיחה וקריאה

```
try {
    $snmp = New-Object -ComObject olePrn.oleSNMP
    $snmp.Open($SnmpHost, $SnmpCommunity, 2, 1000)

    foreach ($singleOid in $Oid) {
        try {
            $raw = ([string]$snmp.Get($singleOid)).Trim()
        }
        catch {
            Write-Log "Label $singleOid could not be read: $('_.Exception.Message)" -Level WARN
            continue
        }
    }
}
```

שורה 204 יוצרת את אובייקט ה-COM ושורה 205 פותחת את ה-session – אותן פקודות בדיוק כמו בסקריפט המקורי, אבל בתוך `try`.

שורות 208–214 מכילות `try/catch` פנימי, לכל OID בנפרד. זו החלטה מכוונת: OID אחד שנכשל לא מבטל את הקריאה של השני. אם היה רק `try` אחד חיצוני, כשל בשרת הראשון היה מסתיר לחלוטין את מצב השני.

`([string]$snmp.Get(...)).Trim()` – המרה מפורשת למחרוזת וחיתוך רווחים, לפני כל ניסיון להשוות.

216–231 Get-WebServerHealth – פענוח הערך

```
$answered++

$value = 0
if (-not [int]::TryParse($raw, [ref]$value)) {
    Write-Log "Label $singleOid returned '$raw', which is not a number. Counting it as
down." -Level WARN
    continue
}

if ($value -eq $HealthyValue) {
    $healthy++
    Write-Log "Label $singleOid = $value (up)."
}
else {
    Write-Log "Label $singleOid = $value (down)." -Level WARN
}
}
```

שורה 216 מקדמת את `$answered`: הגענו לכאן, כלומר התקבלה תשובה – בלי קשר לתוכנה.

שורות 218–222 הן התיקון להשוואת מחרוזת למספר. `[int]::TryParse` מנסה להמיר ומחזיר הצלחה/כישלון בלי לזרוק. ערך שאינו מספר נרשם בלוג ונספר כנפילה – החלטה שמרנית ומודעת, במקום התנהגות מקרית.

שורות 224–230 מבצעות את ההשוואה עצמה ורושמות כל ערך בלוג. זה מה שיאפשר, בדיעבד, להבין למה הסקריפט החליט מה שהחליט.

232–247 Get-WebServerHealth – ניקוי מסודר

```
}
catch {
    Write-Log "SNMP session to $Label ($Snmphost) failed: $($_.Exception.Message)" -Level WARN
}
finally {
    if ($snmp) {
        try {
            $snmp.Close()
            [void][System.Runtime.InteropServices.Marshal]::ReleaseComObject($snmp)
        }
        catch {
            Write-Log "Releasing the SNMP session to $Snmphost failed: $($_.Exception.Message)" -
Level WARN
        }
    }
}
```

ה- `catch` בשורות 233–235 תופס את מה שהפיל את הסקריפט המקורי: כשל בפתיחת ה-session. כאן זה נרשם כאזהרה, והפונקציה ממשיכה להחזיר תוצאה תקינה שאומרת 'לא קיבלנו תשובה'.

ה- `finally` בשורות 236–246 רץ **תמיד** – גם במסלול המוצלח וגם בכושל. הוא סוגר את ה-session ומשחרר את אובייקט ה-COM. בסקריפט המקורי, כשל אחרי `Open` היה מדלג על `Close` ומשאיר אובייקט COM תלוי.

שימו לב שגם הניקוי עצמו עטוף ב- `try/catch`: כשל בשחרור לא אמור להפיל את הסקריפט. ההערה בשורות 238–244 נכתבה אחרי שהבדיקות הראו בדיוק את המקרה הזה.

248–253 Get-WebServerHealth – ערך ההחזרה

```
return [pscustomobject]@{
    Answered = ($answered -gt 0)
    Healthy = $healthy
}
}
```

`[pscustomobject]` עם שני שדות מפורשים. `Answered` הוא `$true` רק אם לפחות OID אחד ענה.

החזרת אובייקט ולא מספר בודד היא מה שמאפשר לקורא להבחין בין המצבים. **שימו לב שכאן אין דליפת פלט** – כל פקודה בפונקציה נכתבת ללוג או מושמת למשתנה, ולכן ערך ההחזרה הוא בדיוק מה שכתוב כאן. זה הלקח מהבאג המרכזי של הגרסה המקורית.

```
function Get-DataCentreState {
    # Serving as soon as one poll finds a web server up, so a single bad reading
    # cannot move the record. Unknown when no poll got an answer at all.
    [OutputType([string])]
    param(
        [Parameter(Mandatory)][string] $Snmphost,
        [Parameter(Mandatory)][string[]] $Oid,
        [Parameter(Mandatory)][string] $Label
    )

    $severAnswered = $false

    for ($attempt = 1; $attempt -le $PollCount; $attempt++) {
        Write-Log "Polling $Label on $Snmphost, attempt $attempt of $PollCount."
        $health = Get-WebServerHealth -Snmphost $Snmphost -Oid $Oid -Label $Label

        if ($health.Answered) {
            $severAnswered = $true

            if ($health.Healthy -gt 0) {
                Write-Log "$Label has $($health.Healthy) web server(s) up."
                return 'Serving'
            }
        }

        if ($attempt -lt $PollCount -and $PollDelaySeconds -gt 0) {
            Start-Sleep -Seconds $PollDelaySeconds
        }
    }
}
```

עוטפת את הקודמת ומתרגמת מספרים למצב. `$severAnswered` בשורה 264 זוכר אם **אי פעם**, באיזו בדיקה שהיא, התקבלה תשובה.

הלולאה בשורות 266–282 היא הגנת ה-flapping. שתי החלטות בתוכה:

יציאה מוקדמת (שורות 273–276): תשובה חיובית אחת מספיקה. אין סיבה להמשיך לשאול כששרת נמצא למעלה, וזה גם מקצר את זמן הריצה במקרה הנפוץ.

השהיה בין בדיקות (שורות 279–281): התנאי `$attempt -lt $PollCount` מונע השהיה מיותרת אחרי הבדיקה האחרונה.

```

if (-not $everAnswered) {
    Write-Log "$Label did not answer any of the $PollCount SNMP polls." -Level WARN
    return 'Unknown'
}

Write-Log "$Label answered, and every web server is down." -Level WARN
return 'Down'
}

```

שתי היציאות שנותרו, וההבחנה ביניהן היא כל מה שהפונקציה נבנתה בשבילו:

Unknown (שורות 284–287) – אף בדיקה לא קיבלה תשובה. איננו יודעים מה מצב מרכז הנתונים. ההחלטה מה לעשות עם זה נדחית לקורא, בשורה 370.

Down (שורות 289–290) – קיבלנו תשובות, וכולן אומרות שהשרתים נפלו. כאן אנחנו **כן** יודעים, והידיעה היא שלילית.

```

function Get-SolarWindsRecord {
    param(
        [Parameter(Mandatory)][string] $Server
    )

    $records = @(Get-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -Name
    $RecordName -RRType A)

    if ($records.Count -eq 0) {
        throw "Zone '$ZoneName' on $Server holds no A record named '$RecordName'."
    }

    if ($records.Count -gt 1) {
        throw "Zone '$ZoneName' on $Server holds $($records.Count) A records named '$RecordName'. This
        script handles a single A record, so the duplicates have to be resolved first."
    }

    return $records[0]
}

```

@(...) סביב הפקודה בשורה 298 מכריח את התוצאה להיות מערך גם כשחוזרת רשומה אחת, כך ש-**Count**. תמיד עובד. בלי זה, רשומה בודדת הייתה מחזירה אובייקט ש-**Count**. עליו מתנהג אחרת.

-RRType A הוא התיקון לשורה 48 של הסקריפט המקורי: רק רשומות A, לא רשומות מסוגים אחרים.

שתי הבדיקות בשורות 300–306 הופכות שני מצבים שהיו קורסים בהודעה לא ברורה להודעות שאומרות **בדיק מה קרה ומה לעשות**. במיוחד מקרה הכפילות: הסקריפט מסרב לנחש איזו רשומה לעדכן, וזו ההתנהגות הנכונה.

```
function Set-SolarWindsRecord {
    # Set-DnsServerResourceRecord needs the untouched record and the edited copy
    # as two separate objects, hence the Clone.
    param(
        [Parameter(Mandatory)][string] $Server,
        [Parameter(Mandatory)][string] $TargetIp
    )

    $current = Get-SolarWindsRecord -Server $Server
    $updated = $current.Clone()
    $updated.RecordData.IPv4Address = [System.Net.IPAddress]::Parse($TargetIp)

    if ($RecordTtlSeconds -gt 0) {
        $updated.TimeToLive = [System.TimeSpan]::FromSeconds($RecordTtlSeconds)
    }
}
```

— `$current.Clone()` בשורה 320 מייצר עותק נפרד. `Set-DnsServerResourceRecord` דורשת שני אובייקטים – המקורי כדי לדעת מה להחליף, והערך כדי לדעת במה. שינוי של אותו אובייקט עצמו לא יעבוד.

שורות 323–325 מטפלות ב-TTL רק אם התבקש במפורש. TTL גבוה הוא בעיה אמיתית ב-failover: רשומה עם TTL של שעה תיכנס לתוקף רק בעוד שעה אצל מי שכבר שמר אותה במטמון.

```
Set-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -OldInputObject $current -
NewInputObject $updated

$after = Get-SolarWindsRecord -Server $Server
$afterIp = $after.RecordData.IPv4Address.IPAddressToString

if ($afterIp -ne $TargetIp) {
    throw "The record was written but still reads $afterIp instead of $TargetIp."
}
}
```

שורה 327 מבצעת את הכתיבה. שורות 329–334 הן **תוספת שאין לה מקבילה בסקריפט המקורי**: קריאה חוזרת של הרשומה ואימות שהערך אכן נשמר.

זה חשוב כי כתיבה ל-DNS יכולה 'להצליח' בלי להיכשל ובכל זאת לא להשפיע – למשל אם השרת מחזיק עותק שאינו בר-כתיבה של ה-Zone. בלי האימות, הסקריפט היה מדווח על הצלחה בזמן שהרשומה נשארה במקומה.

```
try {
    Write-Log "==== Run started for active server $ActiveServer ====="

    if ($ActiveServer -eq $PrimaryDcSolarWindsIp) {
        $localName      = 'PrimaryDC'
        $remoteName     = 'SecondaryDC'
        $snmpHost       = $PrimaryDcSnmpHost
        $oids           = $PrimaryDcWebServerOid
        $localLbIp      = $PrimaryDcLoadBalancerIp
        $remoteLbIp     = $SecondaryDcLoadBalancerIp
    }
    elseif ($ActiveServer -eq $SecondaryDcSolarWindsIp) {
        $localName      = 'SecondaryDC'
        $remoteName     = 'PrimaryDC'
        $snmpHost       = $SecondaryDcSnmpHost
        $oids           = $SecondaryDcWebServerOid
        $localLbIp      = $SecondaryDcLoadBalancerIp
        $remoteLbIp     = $PrimaryDcLoadBalancerIp
    }
    else {
        throw "ActiveServer '$ActiveServer' is neither the PrimaryDC SolarWinds server
        ($PrimaryDcSolarWindsIp) nor the SecondaryDC one ($SecondaryDcSolarWindsIp)."
    }
}
```

כל גוף הסקריפט עטוף ב- `try` אחד גדול שנסגר רק בשורה 405. כל שגיאה בלתי צפויה תיתפס בשורה 406 ותיהפך לקוד יציאה 1.

שורות 340–355 מתרגמות את `$ActiveServer` לשישה משתנים: מיהו 'המקומי', מיהו 'המרוחק', איזו כתובת SNMP לשאול, ואילו OLD-ים. **כאן מרוכזת כל האסימטריה בין שני מרכזי הנתונים** – ומכאן והלאה הקוד זהה לשניהם. זה מה שמחליף את ארבעת הבלוקים הכפולים של הסקריפט המקורי: במקום שני מסלולי קוד נפרדים, מסלול אחד עם שני סטים של ערכים. שורות 356–358 – ה- `else` – הן התיקון לכשל השקט: כתובת שאינה מוכרת **עוצרת את הריצה** במקום לגרום לסקריפט לסיים בהצלחה בלי לעשות דבר.

```
Write-Log "Running for $localName."

$dnsServerAddress = if ($DnsServer) { $DnsServer } else { Resolve-DnsServerAddress }

$record = Get-SolarWindsRecord -Server $dnsServerAddress
$currentIp = $record.RecordData.Ipv4Address.IPAddressToString
Write-Log "$RecordName.$ZoneName currently points at $currentIp with a TTL of $
($record.TimeToLive)."
```

שורה 362 משתמשת ב- `if` כביטוי – תחביר תמציתי שמשמעו 'אם ניתנה כתובת מפורשת, השתמש בה; אחרת חפש'.

שורות 364–366 קוראות את הרשומה ורושמות את מצבה בלוג, כולל ה-TTL. **הקריאה נעשית לפני התשאל** – כך שגם אם משהו ייכשל בהמשך, הלוג כבר מתעד לאן הרשומה הצביעה ברגע שהריצה התחילה.

368–379 ההחלטה על מצב לא ידוע

```
$state = Get-DataCentreState -SnmpHost $snmpHost -Oid $oids -Label $localName

if ($state -eq 'Unknown') {
    if ($SnmpFailureMode -eq 'Unknown') {
        Write-Log "$localName could not be reached over SNMP and SnmpFailureMode is 'Unknown', so
the record is left at $currentIp." -Level ERROR
        exit 2
    }

    Write-Log "$localName could not be reached over SNMP. SnmpFailureMode is 'Down', so it is
treated as a dead data centre." -Level WARN
    $state = 'Down'
}
```

שורה 368 מריצה את התשאל, ושורות 370–378 מטפלות במצב **Unknown** – המקרה שבסקריפט המקורי פשוט הפיל את התהליך.

הבחירה נעשית לפי **\$SnmpFailureMode**, ושתי האפשרויות לגיטימיות:

Unknown – לא נוגעים ברשומה, יוצאים בקוד 2. בטוח יותר כשה-SNMP לא אמין.

Down – מתייחסים לאי-מענה כאל מרכז נתונים מת ומבצעים failover. זו ההתנהגות שהסקריפט המקורי ניסה להשיג. ההיגיון: אם אי אפשר לדבר עם ה-LB, סביר שגם המשתמשים לא מצליחים.

בכל מקרה – **ההחלטה נרשמת בלוג עם הנימוק**.

380–393 בחירת היעד ובדיקת האידמפוטנטיות

```
if ($state -eq 'Serving') {
    $targetIp = $localLbIp
    $targetName = $localName
}
else {
    $targetIp = $remoteLbIp
    $targetName = $remoteName
}

if ($currentIp -eq $targetIp) {
    Write-Log "Nothing to do, the record already points at the $targetName load balancer
($targetIp)."
    exit 0
}
```

שורות 380–387 הן כל לוגיקת ההחלטה, בחמש שורות: **Serving** → מקומי, אחרת → מרוחק.

שורות 389–392 הן המקבילה ל-`-and $ip -ne $LB_x` של הסקריפט המקורי – החלק היחיד שם שהיה נכון מלכתחילה. ההבדל: כאן זו בדיקה **אחת** במקום ארבע, והיא **נרשמת בלוג** במקום לעבור בשקט.

תמיכה ב-WhatIf 394–401

```
$target = "$RecordName.$ZoneName on $dnsServerAddress"
$action = "Repoint from $currentIp to $targetIp, the $targetName load balancer"

if (-not $PSCmdlet.ShouldProcess($target, $action)) {
    Write-Log "WhatIf: would have repointed $RecordName.$ZoneName from $currentIp to $targetIp
($targetName load balancer)."
    exit 0
}
```

`$PSCmdlet.ShouldProcess` הוא המנגנון שמאחורי `-WhatIf`. בהרצה רגילה הוא מחזיר `$true` והקוד ממשיך; עם `-WhatIf` הוא מדפיס מה היה קורה ומחזיר `$false`.

שני המחרוזות בשורות 394–395 הן מה שיוצג: מה המטרה ומה הפעולה. ניסוח ברור כאן הוא ההבדל בין תצוגה מקדימה שימושית לכזו שלא אומרת דבר.

שורה 398 רושמת גם בלוג – ובזכות `-WhatIf:$false` משורה 159, הרישום הזה אכן מגיע לקובץ.

הכתיבה וסיום מוצלח 402–405

```
Set-SolarWindsRecord -Server $dnsServerAddress -TargetIp $targetIp
Write-Log "Repointed $RecordName.$ZoneName from $currentIp to $targetIp, the $targetName load
balancer."
exit 0
}
```

שלוש שורות: כתיבה, רישום, יציאה בקוד 0. `exit 0` מפורש אומר לקורא שהכול הסתיים כשורה.

טיפול בשגיאה כללית 406–411

```
catch {
    Write-Log "Run failed: $($_.Exception.Message)" -Level ERROR
    Write-Log $_.ScriptStackTrace -Level ERROR
    Write-Error -Message $_.Exception.Message -ErrorAction Continue
    exit 1
}
```

גוש ה-`catch` הסוגר. שלוש פעולות נפרדות ומכוונות:

שורה 407 רושמת את ההודעה בלוג. שורה 408 רושמת גם את `ScriptStackTrace` – **באיזו שורה בדיוק נכשל**, מה שהופך דיבוג בדיעבד לאפשרי.

שורה 409 כותבת גם ל-stderr, כי מתזמנים רבים תופסים את הזרם הזה בנפרד. `-ErrorAction Continue` נחוץ כדי לעקוף את `$ErrorActionPreference = 'Stop'` משורה 137 – בלעדיו, `Write-Error` עצמו היה זורק חריגה חדשה מתוך ה-catch.

שורה 410 מחזירה קוד 1.

4. סקריפט ג' – בלי Alteon ובלי שרתי web, שורה אחר שורה

הקובץ `DNS.SetSolarWindsRecordToActiveServer.ps1`, 240 שורות, ב-12 מקטעים רצופים. כאן ההחלטה עצמה שונה: אין load balancer בנתיב ואין תשאול SNMP. הסקריפט מקבל מ-SolarWinds את כתובת השרת שהפך לפעיל, ומפנה אליה את הרשומה.

1-23 הצהרת גרסה ובלוק העזרה – תיאור

```
#Requires -Version 5.1

<#
.SYNOPSIS
    Points the SolarWinds DNS record at the SolarWinds server that has just
    become active. No load balancer, no web servers, no SNMP.

.DESCRIPTION
    This is the Alteon-free variant of DNS.SetSolarWindsRecordToActiveLB.v2.ps1.
    That script asks a Radware load balancer over SNMP how the web servers
    behind it are doing, and picks a load balancer accordingly. This one does
    not ask anything: it is told which SolarWinds server is now active, and
    points the record straight at it.

    Being told is the point. SolarWinds decided which server is active, so
    SolarWinds is the authority on it. Anything the script worked out for
    itself would at best reproduce that answer and at worst contradict it.

    It is meant to run as an alert action on the server that has just become
    active, with that server's address passed in -ActiveServer. Nothing is
    written when the record already holds the wanted address, so it is also
    safe to run on a schedule.
```

אותה הצהרת `#Requires` כמו בשני הסקריפטים הקודמים.

שורות 8–13 מגדירות את ההבדל מול סקריפט ב': שם שואלים load balancer, דרך SNMP, איך שרתי ה-web שמאחוריו מתפקדים, ובוחרים LB לפי התשובה. כאן **לא שואלים כלום** – הסקריפט מקבל את הכתובת של השרת שהפך לפעיל, ומפנה אליו את הרשומה.

שורות 15–17 מנמקות את ההחלטה הזו, והיא עיקר ההבדל בין הגרסה הזו לקודמת שלה: **SolarWinds הוא שהחליט מי השרת הפעיל, ולכן הוא הסמכות על כך**. כל בירור שהסקריפט היה עושה בעצמו – בדיקת זמינות, שאילתה על בריכת HA – היה במקרה הטוב משחזר מידע שכבר קיים, ובמקרה הרע סותר אותו. בירור כזה גם מוסיף דרכים חדשות להיכשל: שאילתה שאינה מתאימה לסכמה, שרת standby שעונה לבדיקת פורט ונחשב בטעות לפעיל.

שורות 19–22 מציינות מתי הסקריפט רץ – כ-action alert על השרת שהפך לפעיל – ואת תכונת האידמפוטנטיות, שבגללה מותר להריץ אותו גם מתוזמן.

.PARAMETER ActiveServer

The address of the SolarWinds server that is now active, and the address the record will be pointed at. It has to match either PrimaryDcSolarWindsIp or SecondaryDcSolarWindsIp, otherwise the script fails instead of quietly doing nothing.

.PARAMETER DnsServer

The DNS server to read the record from and write it back to. It has to hold a writable copy of the zone. When it is left out the script walks the DNS servers configured on this host and takes the first one that answers a ping.

.PARAMETER RecordTtlSeconds

When greater than zero, the TTL written onto the record. A failover only takes effect once the old TTL has expired everywhere, so a low value (30-60) is worth setting here.

ActiveServer (שורות 24–29) הוא ליבת הסקריפט. שימו לב לניסוח: הכתובת הזו היא **גם הזיהוי וגם היעד** – מולה נקבע איזה צד פעיל, ואותה כותבים לרשומה. בגרסה קודמת של הסקריפט היו לכך שני פרמטרים נפרדים; הם הוסרו, כי בלי load balancer בנתיב הם תמיד החזיקו אותו ערך.

DnsServer (שורות 30–34) ו-**RecordTtlSeconds** (שורות 35–39) זהים במשמעותם לסקריפט ב'.

.EXAMPLE

```
.\DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.10.1 -WhatIf
```

Shows what would change without touching DNS.

.EXAMPLE

```
.\DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.20.1
```

.NOTES

Exit codes: 0 done (changed or nothing to change), 1 failed.

Call it with -File, not -Command:

```
powershell.exe -NoProfile -File DNS.SetSolarWindsRecordToActiveServer.ps1 -ActiveServer 10.10.10.1
```

A parameter that fails validation stops the script before it runs, and only -File turns that into exit code 1. Under -Command the caller reads a stale \$LASTEXITCODE instead, so a typo in the arguments looks like a clean run.

The record points at the SolarWinds server itself. If a load balancer sits in front of SolarWinds in your environment, this is the wrong script for it: use DNS.SetSolarWindsRecordToActiveLB.v2.ps1, which picks between the two load balancers on the health of the web servers behind them.

#>

שתי דוגמאות. הראשונה עם `-WhatIf`, הדרך לבדוק את הסקריפט בלי לגעת ב-DNS.

שורות 48–49 מציגות שני קודי יציאה בלבד, 0 ו-1. אין כאן קוד 2: הוא היה שמור למצב 'לא הצלחתי לברר מי פעיל', ובלי שלב בירור המצב הזה לא קיים.

שורות 51–57 חוזרות על דרישת `-File`, מאותה סיבה שבסקריפט ב'.

שורות 59–62 הן הפניה מפורשת: אם יש load balancer מול SolarWinds, **זה הסקריפט הלא נכון** ויש להשתמש בגרסת ה-LB. עדיף לומר זאת בעזרה מאשר להשאיר את הקורא לגלות זאת בדרך הקשה.

הצהרת הפרמטרים

65–90

```
[CmdletBinding(SupportsShouldProcess)]
param(
    [Parameter(Mandatory, Position = 0)]
    [ValidateNotNullOrEmpty()]
    [string] $ActiveServer,

    [ValidateNotNullOrEmpty()]
    [string] $RecordName = 'OurSolar',

    [ValidateNotNullOrEmpty()]
    [string] $ZoneName = 'OurZone',

    [string] $DnsServer,

    [ValidateNotNullOrEmpty()]
    [string] $PrimaryDcSolarWindsIp = '10.10.10.1',

    [ValidateNotNullOrEmpty()]
    [string] $SecondaryDcSolarWindsIp = '10.10.20.1',

    [ValidateRange(0, 86400)]
    [int] $RecordTtlSeconds = 0,

    [string] $LogPath = (Join-Path -Path $env:ProgramData -ChildPath 'SolarWinds\dns-active-
server.log')
)
```

`[CmdletBinding(SupportsShouldProcess)]` מקנה לסקריפט את `-WhatIf`, שיגיע לידי ביטוי בשורה 226.

`$ActiveServer` מוצהר `Mandatory` עם `ValidateNotNullOrEmpty`. **זהו התיקון המלא ל-`$args[0]` של הסקריפט המקורי**: שם, ארגומנט חסר או שגוי גרם לכך ששני תנאי ה-`if` לא התקיימו והסקריפט הסתיים בהצלחה מדומה בלי לעשות דבר. כאן הוא נעצר בשלב קישור הפרמטרים.

`Position = 0` שומר על תאימות מלאה לצורת ההפעלה הישנה: אפשר להעביר את הכתובת בלי לנקוב בשם הפרמטר, בדיוק כמו ל-`$args[0]`.

שאר הפרמטרים – שם הרשומה, ה-`Zone`, שרת ה-DNS, שתי כתובות שרתי ה-SolarWinds, ה-TTL ונתיב הלוג – היו ערכים קבועים בקוד המקורי. `$DnsServer` הוא היחיד בלי `ValidateNotNullOrEmpty`, במכוון: השארתו ריקה מפעילה את האיתור האוטומטי.

שמונה פרמטרים בסך הכול. אין כאן פרמטרים לשיטת זיהוי, לשאילתת HA, לפורט בדיקה או לניסיונות חוזרים – אין מה לזהות ואין מה לנסות שוב, כי התשובה מגיעה מבחוץ.

מצב קפדני וטיפול בשגיאות 91–93

```
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
```

שתי שורות זהות לאלה שבסקריפט ב', והנימוק המלא נמצא שם.

בקצרה: `Set-StrictMode` הופך פנייה למשתנה שלא הוגדר או למאפיין שאינו קיים לשגיאה במקום ל-`$null` שקט, ו-`$ErrorActionPreference = 'Stop'` הופך שגיאות לא-מפסיקות למפסיקות – מה שגורם ל-`try/catch` לעבוד באמת.

Write-Log 94–120

```
function Write-Log {
    param(
        [Parameter(Mandatory)]
        [AllowEmptyString()]
        [string] $Message,

        [ValidateSet('INFO', 'WARN', 'ERROR')]
        [string] $Level = 'INFO'
    )

    $line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
    Write-Information -MessageData $line -InformationAction Continue

    # -WhatIf:$false so that a -WhatIf run still leaves a trace of what it
    # decided; the log is a record, never one of the changes being previewed.
    try {
        $directory = Split-Path -Path $script:LogPath -Parent
        if ($directory -and -not (Test-Path -LiteralPath $directory)) {
            New-Item -Path $directory -ItemType Directory -Force -WhatIf:$false -Confirm:$false | Out-Null
        }
        Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -WhatIf:$false -Confirm:$false
    }
    catch {
        Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
    }
}
```

זהה בתו לפונקציה שבסקריפט ב', שורות 139–164 שם, למעט נתיב הלוג שנקבע בפרמטר.

שתי נקודות ששוות חזרה: `Write-Information` ולא `Write-Host`, כדי שמתזמן יוכל לתפוס את הזרם; ו-`-WhatIf:$false` בשורות 112 ו-114, כדי שגם הרצת תצוגה מקדימה תשאיר תיעוד של מה שהוחלט.

נתיב ברירת המחדל הוא `dns-active-server.log` ולא `dns-failover.log`. **מכונן:** שני הסקריפטים יכולים לרוץ באותה מכונה, ואסור שיכתבו לאותו קובץ ויבלבלו את התחקיר.

איתור שרת DNS 121–143

```
function Resolve-DnsServerAddress {
    # Walks the DNS servers configured on this host and returns the first one
    # that answers a ping. Only adapters with IP enabled are looked at, so
    # disconnected or disabled NICs cannot contribute a stale address.
    [OutputType([string])]
    param()

    $adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -Filter 'IPEnabled = True'

    foreach ($adapter in $adapters) {
        foreach ($address in $adapter.DNSServerSearchOrder) {
            if (Test-Connection -ComputerName $address -Count 1 -Quiet -ErrorAction SilentlyContinue) {
                Write-Log "Using DNS server $address, configured on '$($adapter.Description)'."
                return [string]$address
            }

            Write-Log "DNS server $address did not answer a ping, trying the next one." -Level WARN
        }
    }

    throw 'None of the DNS servers configured on this host answered a ping. Pass -DnsServer to name one explicitly.'
}
```

זהה בתו לשורות 166–187 בסקריפט ב'.

בקצרה: `Get-CimInstance` עם `-Filter 'IPEnabled = True'` כדי שכרטיס מנוטרל לא יתרום כתובת ישנה; `Test-Connection -Quiet` בתוך `if` כדי שלא תהיה דליפת פלט ולא `return` לא-מותנה – שני הבאגים של `dns_ip` המקורית; ושגיאה מפורשת בשורה 141 אם אף שרת לא ענה.

```

function Get-SolarWindsRecord {
    param(
        [Parameter(Mandatory)][string] $Server
    )

    $records = @(Get-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -Name
$RecordName -RRType A)

    if ($records.Count -eq 0) {
        throw "Zone '$ZoneName' on $Server holds no A record named '$RecordName'."
    }

    if ($records.Count -gt 1) {
        throw "Zone '$ZoneName' on $Server holds $($records.Count) A records named '$RecordName'. This
script handles a single A record, so the duplicates have to be resolved first."
    }

    return $records[0]
}

function Set-SolarWindsRecord {
    # Set-DnsServerResourceRecord needs the untouched record and the edited copy
    # as two separate objects, hence the Clone.
    param(
        [Parameter(Mandatory)][string] $Server,
        [Parameter(Mandatory)][string] $TargetIp
    )

    $current = Get-SolarWindsRecord -Server $Server
    $updated = $current.Clone()
    $updated.RecordData.IPv4Address = [System.Net.IPAddress]::Parse($TargetIp)

    if ($RecordTtlSeconds -gt 0) {
        $updated.TimeToLive = [System.TimeSpan]::FromSeconds($RecordTtlSeconds)
    }

    Set-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -OldInputObject $current -
NewInputObject $updated

    $after = Get-SolarWindsRecord -Server $Server
    $afterIp = $after.RecordData.IPv4Address.IPAddressToString

    if ($afterIp -ne $TargetIp) {
        throw "The record was written but still reads $afterIp instead of $TargetIp."
    }
}

```

`Get-SolarWindsRecord` ו-`Set-SolarWindsRecord` זהות בתו לשורות 293–335 בסקריפט ב'.

בקצרה: קריאה עם `RRType A` - וסירוב מפורש כשיש כפילות או כשאין רשומה; כתיבה עם `Clone()` כי הפקודה דורשת שני אובייקטים נפרדים, TTL אופציונלי, ואימות חוזר בשורות 178–185 שהערך אכן נשמר.

ששני הסקריפטים חולקים את הקוד הזה מילה במילה הוא מכונן: הם מתנהגים זהה מול ה-DNS, וכל ההבדל ביניהם הוא בדרך שבה הם מחליטים לאן להצביע.

תחילת הגוף, ניקוי הקלט ואימות הכתובת 188–210

```
try {
    Write-Log "==== Run started for active server $ActiveServer ====="

    # A SolarWinds alert action fills this in from a macro, and a macro can
    # hand over its value padded with whitespace. Trimming here means a stray
    # space cannot stop a failover; without it the address matches neither
    # server and the run fails for no good reason.
    $ActiveServer = $ActiveServer.Trim()

    # The address is checked against the two servers the script knows rather
    # than used as given, so a typo in the alert action cannot point the record
    # at something that was never a SolarWinds server.
    if ($ActiveServer -eq $PrimaryDcSolarWindsIp) {
        $activeName = 'PrimaryDC'
    }
    elseif ($ActiveServer -eq $SecondaryDcSolarWindsIp) {
        $activeName = 'SecondaryDC'
    }
    else {
        throw "ActiveServer '$ActiveServer' is neither the PrimaryDC SolarWinds server
        ($PrimaryDcSolarWindsIp) nor the SecondaryDC one ($SecondaryDcSolarWindsIp)."
    }

    Write-Log "$activeName is active, so the record should point at $ActiveServer."
```

כל הגוף עטוף ב- `try` אחד שנסגר בשורה 228.

שורות 191–196 מטפלות במקרה שעולה דווקא מהשימוש האמיתי: alert action של SolarWinds ממלא את הפרמטר ממאקרו, ומאקרו עלול למסור את ערכו עם רווחים מסביב. בלי `.Trim()` הכתובת לא הייתה תואמת לאף אחד משני השרתים, והריצה הייתה נכשלת בלי סיבה אמיתית – רווח בודד היה מונע failover. התקלה הזו התגלתה בבדיקה, לא בקריאת הקוד.

ההערה בשורות 198–200 מסבירה את ההחלטה בשורות 201–209: הכתובת שהתקבלה **נבדקת מול שתי הכתובות שהסקריפט מכיר, ולא נלקחת כמות שהיא**. אפשר היה פשוט לכתוב לרשומה את מה שהועבר – הרי זה בדיוק מה שמבקשים – אבל אז טעות הקלדה ב-alert action הייתה מפנה את כל הארגון לכתובת שמעולם לא הייתה שרת SolarWinds.

ה- `else` בשורות 207–209 הוא הסירוב המפורש. זה כל מה שנשאר מהמיפוי הגדול שהיה כאן בגרסאות הקודמות: בסקריפט ב' צריך היה לגזור מהכתובת שישה משתנים – מקומי, מרוחק, כתובת SNMP ו-OID-ים. כאן צריך רק את שם הצד, לצורך הלוג.

```

$dnsServerAddress = if ($DnsServer) { $DnsServer } else { Resolve-DnsServerAddress }

$record = Get-SolarWindsRecord -Server $dnsServerAddress
$currentIp = $record.RecordData.Ipv4Address.IPAddressToString
Write-Log "$RecordName.$ZoneName currently points at $currentIp with a TTL of $
($record.TimeToLive)."

if ($currentIp -eq $ActiveServer) {
    Write-Log "Nothing to do, the record already points at the $activeName server ($ActiveServer)."
    exit 0
}

$target = "$RecordName.$ZoneName on $dnsServerAddress"

```

שורה 213 בוחרת את שרת ה-DNS: מפורש אם ניתן, אחרת איתור אוטומטי.

שורות 215–217 קוראות את הרשומה ורושמות את מצבה בלוג **לפני** כל שינוי, כך שגם אם משהו ייכשל בהמשך, הלוג כבר מתעד לאן הרשומה הצביעה בתחילת הריצה.

שורות 219–222 הן בדיקת 'אין מה לעשות'. **זו הדרך שבה הסקריפט בטוח להרצה חוזרת** – במצב יציב הוא קורא, משווה, רושם שורה אחת ויוצא, בלי לגעת ב-DNS. זו המקבילה ל-`$LB_x -ne $ip -and` של הסקריפט המקורי, החלק היחיד שם שהיה נכון מלכתחילה, אלא שכאן הבדיקה נעשית פעם אחת ונרשמת בלוג במקום לעבור בשקט ארבע פעמים.

```

$action = "Repoint from $currentIp to $ActiveServer, the $activeName server"

if (-not $PSCmdlet.ShouldProcess($target, $action)) {
    Write-Log "WhatIf: would have repointed $RecordName.$ZoneName from $currentIp to $ActiveServer
($activeName server)."
    exit 0
}

Set-SolarWindsRecord -Server $dnsServerAddress -TargetIp $ActiveServer
Write-Log "Repointed $RecordName.$ZoneName from $currentIp to $ActiveServer, the $activeName
server."
exit 0
}
catch {

```

`$PSCmdlet.ShouldProcess` בשורה 226 הוא המנגנון שמאחורי `-WhatIf`: בהרצה רגילה הוא מחזיר `$true` והקוד ממשיך, ועם `-WhatIf` הוא מציג את שתי המחרוזות משורות 224–225 ומחזיר `$false`.

שורה 227 רושמת גם בלוג, ובזכות `-WhatIf:$false` שראינו קודם, הרישום הזה אכן מגיע לקובץ.

שורות 231–233: כתיבה, רישום ויציאה בקוד 0.

```
Write-Log "Run failed: $($_.Exception.Message)" -Level ERROR
Write-Log $_.ScriptStackTrace -Level ERROR
Write-Error -Message $_.Exception.Message -ErrorAction Continue
exit 1
}
```

זהה לסקריפט ב'. שורה 237 רושמת את ההודעה, שורה 238 את ה- `ScriptStackTrace` – באיזו שורה בדיוק נכשל – ושורה 239 כותבת גם ל-`stderr`, כי מתזמנים רבים תופסים את הזרם הזה בנפרד. `ErrorAction Continue` – שם נחוץ כדי לעקוף את `$ErrorActionPreference = 'Stop'`; בלעדיו `Write-Error` עצמו היה זורק חריגה חדשה מתוך ה-`catch`.

שורה 240 מחזירה קוד 1, ובזה נסגר ה- `try` שנפתח בשורה 188.

5. השוואה, מצב הבדיקות ומה לאמת לפני הרצה

איך שלושת הסקריפטים מחליטים

היבט	א' – מקורי	ב' – מתוקן	ג' – בלי Alteon
מקור ההחלטה	LB ל-SNMP	LB ל-SNMP	הכתובת שהועברה מ-SolarWinds
תלות ביצרן	Radware	Radware	אין
יעד הרשומה	LB	LB	שרת ה-SolarWinds הפעיל
LB לא זמין	הסקריפט קורס, אין failover	החלטה לפי <code>-SnmpFailureMode</code>	לא רלוונטי – אין LB
חוסר ידיעה	נראה כמו נפילה	מצב <code>Unknown</code> נפרד	לא קיים – התשובה מגיעה מבחון
הגנת flapping	אין	<code>-PollCount</code>	לא נדרשת – אין תשאל
לוג	<code>Write-Host aaaaa</code>	קובץ עם חותמות זמן	קובץ נפרד משלו
ולידציה	אין	<code>param()</code> מלא	<code>param()</code> מלא
הנחות לא מאומתות	הערך 0	הערך 0	אין
<code>-WhatIf</code>	אין	יש	יש
קודי יציאה	אין	2 / 1 / 0	1 / 0

מתי להשתמש באיזה

סקריפט ב' מתאים כשה-LB הוא חלק אמיתי מהארכיטקטורה ומצב שרתי ה-web הוא האות הנכון. הוא רואה תקלה שסקריפט ג' לא יראה: שרתי web שנפלו בזמן ששרת ה-SolarWinds עצמו חי ובריא.

סקריפט ג' מתאים כשאין LB בנתיב, והשאלה היחידה היא לאיזה שרת SolarWinds להפנות. הוא גם קל בהרבה לתפעול: אין OLD-ים להתחזק, אין community string לנהל, ואין תלות ביצרן.

למה סקריפט ג' לא מברר בעצמו מי פעיל. גרסה מוקדמת שלו ידעה גם לתשאל את בריכת ה-HA של SolarWinds, וגם לבדוק איזה שרת עונה בפורט. שתי היכולות הוסרו.

הנימוק: SolarWinds הוא שהחליט מי השרת הפעיל, ולכן הוא הסמכות על כך. בירור עצמאי היה במקרה הטוב משחזר מידע שכבר עובר ב-alert action, ובמקרה הרע סותר אותו – שאילתה שאינה מתאימה לסכמת ה-Orion, או שרת standby שעונה לבדיקת פורט ונחשב בטעות לפעיל. הסרתן קיצרה את הסקריפט ב-206 שורות והעלימה את ההנחה הלא-מאומתת היחידה שהיה בו.

מצב הבדיקות

שני הסקריפטים המתוקנים נבדקו מול תחליפים לפקודות Windows ו-SolarWinds, כך שהלוגיקה נבדקת בלי לגעת ב-DNS או בצידוד אמיתי:

סקריפט	תרחישים	מה נבדק
ב' – מתוקן	17, כולם עוברים	failover לשני הכיוונים, שרת בודד למעלה, רשומה כבר נכונה, כשל SNMP בשני המצבים, רשומות כפולות, היעדר רשומה, ערך לא תקין, <code>-WhatIf</code> , ולידציות
ג' – בלי Alteon	16, כולם עוברים	שני הכיוונים, רשומה כבר נכונה, ארגומנט מיקומי כמו <code>\$args[0]</code> , כתובת לא מוכרת, רשומות כפולות, היעדר רשומה, TTL, שרת DNS מפורש, <code>-WhatIf</code> , ולידציות, ודחיית הפרמטרים שהוסרו

מה הבדיקות לא מוכיחות. הן מאמתות את הלוגיקה, לא את ההתנהגות מול צידוד Radware אמיתי או מול שרת DNS אמיתי. הן אינן תחליף להרצה מבוקרת בסביבת מעבדה.

ההנחה שחייבת אימות

בסקריפטים א' ו-ב': הערך 0 מ-SNMP פירושו "השרת למעלה".

זהו הערך שהגרסה המקורית השוותה מולו, והוא נשמר כברירת מחדל. אם בפועל 0 אינו מסמן שרת תקין על ה-LB שלכם, כל ההחלטות מתהפכות – הסקריפט יבצע failover כששרתים תקינים ויישאר במקום כשהם נפלו. הבדיקה: `snmpwalk` על ה-OID מול שרת תקין ומול שרת מושבת. הפרמטר `-HealthyValue` מאפשר לשנות בלי לגעת בקוד.

בסקריפט ג' אין הנחה כזו. הוא אינו מתשאל SNMP ואינו שואל את SolarWinds דבר, אלא מקבל את הכתובת ובודק אותה מול שתי כתובות ידועות. כל מה שהוא עושה ניתן לאימות מקריאת הקוד.

מה עוד לבדוק לפני הרצה אמיתית

1. שרת ה-DNS הוא **Primary של ה-Zone**. כתיבה לעותק שאינו בר-כתיבה תיכשל. במקרה של ספק עדיף להעביר `-DnsServer` מפורש.
2. ה-TTL של הרשומה. TTL של שעה פירושו ש-failover ייכנס לתוקף רק בעוד שעה. ערך של 30 עד 60 שניות מתאים לרשומה שנועדה להתחלף.
3. מחזורת ה-SNMP community (בסקריפטים א' ו-ב' בלבד). `public` הוא ברירת מחדל, לא המלצה.
4. להריץ פעם אחת עם `-WhatIf` ולקרוא את הלוג לפני שנותנים לסקריפט לכתוב ל-DNS ייצור.
5. להפעיל עם `-File` ולא עם `-Command`, אחרת טעות בפרמטרים תיראה כמו ריצה מוצלחת.

מסמך זה מתעד את הסקריפטים שתחת `Scripts/` בריפוזיטורי: `DNS.SetSolarWindsRecordToActiveLB.ps1`, `DNS.SetSolarWindsRecordToActiveLB.v2.ps1` ו- `DNS.SetSolarWindsRecordToActiveServer.ps1`. הקוד המוצג נשלף אוטומטית מקבצי המקור לפי טווחי שורות, כך שהוא זהה לקוד שבריפו. כתובות ה-IP, שם ה-Zone ושם הרשומה המופיעים כאן הם ערכי ברירת המחדל שבקוד, וניתן לשנות כל אחד מהם בפרמטר בלי לערוך את הסקריפטים.